AI & Machine Learning in Business — Complete Exam Notes

Comprehensive Study & Preparation Guide | Theory, Math Problems & Function Directory

EXAM BLUEPRINT & BREAKDOWN

Part A: Proctored MCQ Exam (40 Marks)

- Library & Function Identification
- Core AI/ML Concepts & Terminology
- Prompting Framework Rules
- RAG Architecture Components

Part B: Theoretical & Computational (60 Marks)

- 4 Questions × 5 Marks (Short Answers / Conceptual)
- 2 Questions × 10 Marks (Detailed Workflows / Evaluation)
- 1 Question × 20 Marks (Comprehensive RAG / ML Numerical)

Module 1: Demystifying AI & Machine Learning

1. Conceptual Disambiguation

- **Artificial Intelligence (AI):** The overarching umbrella encompassing any technique that enables computers to mimic human intelligence, logic, or decision-making.
- **Machine Learning (ML):** A subset of AI where systems learn statistical patterns from data to improve performance on tasks without being explicitly programmed.
- **Deep Learning (DL):** A subset of ML utilizing multi-layered Artificial Neural Networks (ANN) to process complex, high-dimensional unstructured data.
- **Generative AI (GenAI):** A specialized subset of DL focused on creating new synthetic content (text, code, images, audio) based on learned probabilistic distributions.

2. The Enterprise AI Value Chain

The enterprise ecosystem consists of four sequential layers:

1. **Hardware Layer:** Compute infrastructure (GPUs, TPUs) providing massive parallel computing capabilities.
2. **Data & Storage Infrastructure:** Vector databases, data lakes, and ETL pipelines supplying clean data.
3. **Foundation Models:** Base Large Language Models (LLMs) and multi-modal models (e.g., Llama 3, GPT-4, Claude).
4. **Application Layer:** Enterprise software, automated agents, and user interfaces driving direct ROI (e.g., dynamic pricing, recommendation engines, automated SRE incident management).

Module 2: Data Strategy, Libraries & Infrastructure

1. Data Lifecycle & Classification

- **Data Lifecycle:** Ingestion → Storage & Processing → Exploratory Data Analysis (EDA) & Cleaning → Model Training/Retrieval → Deployment & Monitoring.
- **Structured Data:** Highly organized tabular data residing in relational databases or CSV files (e.g., transaction records, customer metrics).
- **Unstructured Data:** Non-tabular, free-form content accounting for >80% of enterprise data (e.g., PDF reports, logs, customer support chats, images).

2. Standard Library & Utility Identification Directory

You must be able to recognize these essential Python libraries and built-in modules on the exam:

Library / Function	Primary Purpose & Usage
`os`	Provides functions for interacting with the operating system, resolving relative/absolute file paths, and reading environment configurations.
`json`	Parses raw JSON strings into Python dictionaries and serializes Python objects into structured JSON strings (crucial for tool execution schemas).
`re`	Executes regular expression pattern matching, string extraction, and text cleaning on unstructured documents or log files.
`dotenv` (`load_dotenv`)	Loads key-value environment variables from a local `.env` file into system memory (`os.getenv`) for secure configuration.

Module 3: Predictive Analytics & Traditional Machine Learning

1. Learning Paradigms

- **Supervised Learning:** Training models on explicit input-output pairs (X, y).
  - *Regression:* Predicts continuous numerical targets (e.g., revenue forecasting, dynamic pricing).
  - *Classification:* Predicts discrete categorical labels (e.g., customer churn prediction, fraud detection, SRE log severity).
- **Unsupervised Learning:** Discovering inherent groupings or hidden patterns in unlabeled data (X only).
  - *Clustering (K-Means):* Groups observations based on metric distance (e.g., RFM customer segmentation).

2. Model Evaluation Metrics

Understanding trade-offs in classification matrices:

Metric	Mathematical Formula	Business Significance
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Overall correct predictions. Misleading in imbalanced datasets.
Precision	$TP / (TP + FP)$	Measures exactness. High Precision minimizes False Positives (e.g., avoiding false fraud alerts).
Recall (Sensitivity)	$TP / (TP + FN)$	Measures completeness. High Recall minimizes False Negatives (e.g., catching all critical security threats).

Module 4: Generative AI, Prompting & Agent Frameworks

1. Prompt Engineering Frameworks

- **Zero-Shot Prompting:** Querying the LLM directly without providing prior examples or explicit structural demonstrations.
- **Few-Shot Prompting:** Assigning a strict persona, output schema, and providing one or more input-output target examples to guide generation format and behavior.

- **Chain-of-Thought (CoT) Prompting:** Instructing the LLM to explicitly break down complex problems into step-by-step intermediate reasoning stages before producing a final answer.
- **ReAct Pattern (Reason + Act):** An agentic framework where the LLM iteratively generates a reasoning step (Thought) and executes an external tool call (Action) to solve multi-step workflows.

2. RAG (Retrieval-Augmented Generation) Architecture

RAG grounds LLM responses in proprietary enterprise documents through a 4-step offline/online pipeline:

1. **Document Ingestion & Chunking:** Splitting unstructured documents into smaller segments using defined chunk sizes and character overlaps to preserve local semantics.
2. **Vector Embedding Generation:** Passing text chunks through neural encoders to create dense vector representations.
3. **Vector Indexing & Similarity Search:** Computing vector distance metrics (such as Dot Product or Cosine Similarity) between a query vector and indexed chunk embeddings to retrieve the top-k relevant contexts.
4. **Grounded Synthesis:** Injecting retrieved context blocks into the LLM prompt, forcing the model to generate factual, cited responses without hallucination.

3. Core AI/ML & RAG Library Directory

Master these specific library calls and class methods for Part A identification:

Library / Method	Function & Execution Role
`ollama`	Interface library for executing local open-weight Large Language Models.
`ollama.chat()`	Executes an interactive prompt payload taking arguments like `model`, `messages` array, and system `options` (e.g., `temperature`).
`SentenceTransformer`	Class from `sentence_transformers` used to initialize local transformer-based embedding models (e.g., `all-MiniLM-L6-v2`).
`SentenceTransformer.encode()`	Converts list of text strings into normalized multi-dimensional NumPy vector arrays (`convert_to_numpy=True`).
`numpy.dot()`	Calculates the mathematical dot product between vector arrays to compute similarity scores across embeddings.
`numpy.argsort()`	Sorts vector similarity array scores; combined with slicing `[::-1]` `[:top_k]` to extract indices of top matching chunks.

Module 5: Computational & Problem-Solving Mastery

Problem Type 1: Text Chunking & Indexing Boundaries

Scenario: You are chunking an enterprise Standard Operating Procedure document of total length $L = 1200$ characters. You set a fixed chunk size of $C = 500$ characters and an overlap of $O = 100$ characters.

Step-by-step Solution:

- **Step Size (Stride):** $S = C - O = 500 - 100 = 400$ characters.
- **Chunk 1 Range:** Index 0 to 500.
- **Chunk 2 Range:** Starts at $0 + 400 = 400$. Index 400 to 900 (Overlaps indices 400-500 with Chunk 1).
- **Chunk 3 Range:** Starts at $400 + 400 = 800$. Index 800 to 1200.
- **Total Chunks Produced:** 3 chunks.

Problem Type 2: Vector Similarity & Retrieval Ranking

Scenario: Given a normalized user query embedding vector $q = [0.6, 0.8]$ and three indexed document chunk embeddings:

$$d_1 = [0.8, 0.6], \quad d_2 = [0.0, 1.0], \quad d_3 = [0.5, 0.5]$$

Step-by-step Solution: Compute dot product dot score $S(q, d) = (q_1 \times d_1) + (q_2 \times d_2)$:

- $S(q, d_1) = (0.6 \times 0.8) + (0.8 \times 0.6) = 0.48 + 0.48 = 0.96$
- $S(q, d_2) = (0.6 \times 0.0) + (0.8 \times 1.0) = 0.00 + 0.80 = 0.80$
- $S(q, d_3) = (0.6 \times 0.5) + (0.8 \times 0.5) = 0.30 + 0.40 = 0.70$

Ranking & Top Selection: Higher score indicates higher semantic similarity. If $top_k = 2$, the system retrieves **Chunk 1 (0.96)** first, followed by **Chunk 2 (0.80)**.

Problem Type 3: Model Precision/Recall Trade-off Analysis

Scenario: An automated IT SRE monitoring system evaluates 100 service incidents. Out of 20 actual critical failures (Positives), the system correctly flags 15 (True Positives), misses 5 (False Negatives), and incorrectly flags 10 normal operations as critical failures (False Positives).

Step-by-step Solution:

- $TP = 15, \quad FN = 5, \quad FP = 10, \quad TN = 70$
- **Precision:** $TP / (TP + FP) = 15 / (15 + 10) = 15 / 25 = 0.60$ (60%)
- **Recall:** $TP / (TP + FN) = 15 / (15 + 5) = 15 / 20 = 0.75$ (75%)
- **Business Impact Analysis:** In SRE operations, high Recall is prioritized to ensure zero critical outages go undetected, accepting slightly higher False Positives.